```
// Java program to find first
// element of a Stream in Java

import java.util.*;
import java.util.stream.*;

public class GFG {

    // Function to find the
    // first_elements in a Stream
    public static <T> T
    firstElementInStream(Stream<T> stream)
    {
        T first_element
            = stream

                // reduce() method reduces the Stream
                // to a single element, which is first.
                .reduce((first, second) -> first)

                // if stream is empty
                // null is returned
                .orElse(null);

        return first_element;
    }

    // Driver code
```

```
    public static void main(String[] args)
    {

        Stream<String> stream
            = Stream.of("Geek_First", "Geek_2",
                        "Geek_3", "Geek_4",
                        "Geek_Last");

        // Print the first element of a Stream
        System.out.println(
            "First Element: "
            + firstElementInStream(stream));
    }
}

// Java program to find first
// element of a Stream in Java

import java.util.*;
import java.util.stream.*;

public class GFG {

    // Function to find the
    // first_elements in a Stream
    public static <T> T
    firstElementInStream(Stream<T> stream)
    {
```

```
    T first_element
        = stream

            // findFirst() method returns
            // the first element of stream
            .findFirst()

            // if stream is empty
            // null is returned
            .orElse(null);

    return first_element;
}

// Driver code
public static void main(String[] args)
{

    Stream<String> stream
        = Stream.of("Geek_First", "Geek_2",
                "Geek_3", "Geek_4",
                "Geek_Last");

    // Print the first element of a Stream
    System.out.println(
        "First Element: "
        + firstElementInStream(stream));
}
```

```
}

// Java program to find last
// element of a Stream in Java using stream.reduce() method

import java.util.*;
import java.util.stream.*;

public class GFG {

    // Function to find the
    // last_elements in a Stream
    public static <T> T
    lastElementInStream(Stream<T> stream)
    {
        T last_element
            = stream

                // reduce() method reduces the Stream
                // to a single element, which is last.
                .reduce((first, second) -> second)

                // if stream is empty
                // null is returned
                .orElse(null);

        return last_element;
    }
```

```
    // Driver code
    public static void main(String[] args)
    {

        Stream<String> stream
            = Stream.of("Geek_First", "Geek_2",
                        "Geek_3", "Geek_4",
                        "Geek_Last");

        // Print the last element of a Stream
        System.out.println(
            "Last Element: "
            + lastElementInStream(stream));
    }
}

// Java program to find last
// element of a Stream using stream.skip() method

import java.util.*;
import java.util.stream.*;

public class GFG {

    // Function to find the
    // last_elements in a Stream
    public static <T> T
```

```
lastElementInStream(Stream<T> stream, int N)
{

    T last_element
        = stream

            // This returns a stream after
            // removing first N-1 elements
            // resultant stream will contain
            // only single element
            .skip(N - 1)

            // findFirst() method return
            // the first element of
            // newly generated stream
            .findFirst()

            // if stream is empty
            // null is returned
            .orElse(null);

    return last_element;
}

// Driver code
public static void main(String[] args)
{
```

```
        Stream<String> stream
            = Stream.of("Geek_First", "Geek_2",
                        "Geek_3", "Geek_4",
                        "Geek_Last");

        int N = 5;

        // Print the last element of a Stream
        System.out.println(
            "Last Element: "
            + lastElementInStream(stream, N));
    }
}

import java.util.Arrays;
import java.util.Collections;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7);
        // Get the first element of the reversed stream
        Integer firstElementOfReversedStream = numbers.stream()
                .sorted(Collections.reverseOrder())
                .findFirst()
                .orElse(null);
       // Handle the case where the stream might be empty
        System.out.println("First element of the reversed stream: " + firstElementOfReversedStream);
```

```
    }
}

// Java program to find the duplicate
// elements in a Stream using Set

import java.util.*;
import java.util.stream.*;

public class GfG {

    // Function to find the
    // duplicates in a Stream
    public static <T> Set<T>
    findDuplicateInStream(Stream<T> stream)
    {

        // Set to store the duplicate elements
        Set<T> items = new HashSet<>();

        // Return the set of duplicate elements
        return stream

            // Set.add() returns false
            // if the element was
            // already present in the set.
            // Hence filter such elements
            .filter(n -> !items.add(n))
```

```
        // Collect duplicate elements
        // in the set
        .collect(Collectors.toSet());
    }

    // Driver code
    public static void main(String[] args)
    {

        // Initial stream
        Stream<Integer> stream
            = Stream.of(5, 13, 4,
                        21, 13, 27,
                        2, 59, 59, 34);

        // Print the found duplicate elements
        System.out.println(
            findDuplicateInStream(stream));
    }
}

// Java program to find the duplicate
// elements in a Stream using Collectors.groupingBy()

import java.util.*;
import java.util.stream.*;
import java.util.function.Function;
```

```
public class GfG {

    // Function to find the
    // duplicates in a Stream
    public static <T> Set<T>
    findDuplicateInStream(Stream<T> stream)
    {

        // Return the set of duplicate elements
        return stream

            // Group the elements along
            // with their frequency in a map
            .collect(
                Collectors.groupingBy(
                    Function.identity(),
                    Collectors.counting()))

            // Convert this map into a stream
            .entrySet()
            .stream()

            // Check if frequency > 1
            // for duplicate elements
            .filter(m -> m.getValue() > 1)

            // Find such elements
```

```
        .map(Map.Entry::getKey)

        // And Collect them in a Set
        .collect(Collectors.toSet());
    }

    // Driver code
    public static void main(String[] args)
    {

        // Initial stream
        Stream<Integer> stream
            = Stream.of(5, 13, 4,
                        21, 13, 27,
                        2, 59, 59, 34);

        // Print the found duplicate elements
        System.out.println(
            findDuplicateInStream(stream));
    }
}

// Java program to find the duplicate
// elements in a Stream
// using Collections.frequency()

import java.util.*;
import java.util.stream.*;
```

```
public class GfG {

    // Function to find the
    // duplicates in a Stream
    public static <T> Set<T>
    findDuplicateInStream(List<T> list)
    {
        // Return the set of duplicate elements
        return

            // Get the stream from the list
            list.stream()

                // Count the frequency of each element
                // and filter the elements
                // with frequency > 1
                .filter(i -> Collections.frequency(list, i) > 1)

                // And Collect them in a Set
                .collect(Collectors.toSet());
    }

    // Driver code
    public static void main(String[] args)
    {

        // Initial stream
```

```
        List<Integer> list
            = Arrays.asList(5, 13, 4,
                            21, 13, 27,
                            2, 59, 59, 34);

        // Print the found duplicate elements
        System.out.println(
            findDuplicateInStream(list));
    }
}

// Java program to count occurrences
// of a character using Stream

import java.util.stream.*;

class GFG {

    // Method that returns the count of the given
    // character in the string
    public static long count(String s, char ch)
    {

        return s.chars()
            .filter(c -> c == ch)
            .count();
    }
```

```
    // Driver method
    public static void main(String args[])
    {
        String str = "geeksforgeeks";
        char c = 'e';
        System.out.println(count(str, c));
    }
}

// Java program to reverse elements
// of a parallel Stream

import java.util.*;
import java.util.stream.*;

class GFG {

    // Generic function to reverse
    // the elements of the parallel stream
    public static <T> Collector<T, ?, Stream<T> > reverseStream()
    {
        return Collectors
            .collectingAndThen(Collectors.toList(),
                               list -> {
                                   Collections.reverse(list);
                                   return list.stream();
                               });
    }
```

```
    // Driver code
    public static void main(String[] args)
    {

        // Get the parallel stream
        List<Integer> lists = Arrays.asList(11, 22, 33, 44);
        Stream<Integer> stream = lists.parallelStream();

        // Reverse and print the elements
        stream.collect(reverseStream())
            .forEach(System.out::println);
    }
}

// Java program to reverse elements
// of a parallel Stream

import java.util.*;
import java.util.stream.*;

class GFG {

    // Generic function to reverse
    // the elements of the parallel stream
    public static <T> Iterator<T> reverseStream(Stream<T> stream)
    {
        return stream
```

```
        .collect(Collectors
                .toCollection(LinkedList::new))
        .descendingIterator();
    }

    // Driver code
    public static void main(String[] args)
    {

        // Get the parallel stream
        List<Integer> lists = Arrays.asList(11, 22, 33, 44);
        Stream<Integer> stream = lists.parallelStream();

        // Reverse and print the elements
        Iterator<Integer> reverse = reverseStream(stream);
        reverse.forEachRemaining(System.out::println);
    }
}

// Java program to reverse elements
// of a parallel Stream

import java.util.*;
import java.util.stream.*;

class GFG {

    // Generic function to reverse
```

```
    // the elements of the parallel stream
    public static <T> Stream<T> reverseStream(Stream<T> stream)
    {
        return stream
            .collect(
                Collector.of(
                    () -> new ArrayDeque<T>(), ArrayDeque::addFirst, (a, b) -> {
                        b.addAll(a);
                        return b;
                    }))
            .stream();
    }

    // Driver code
    public static void main(String[] args)
    {

        // Get the parallel stream
        List<Integer> lists = Arrays.asList(11, 22, 33, 44);
        Stream<Integer> stream = lists.parallelStream();

        // Reverse and print the elements
        reverseStream(stream)
            .forEach(System.out::println);
    }
}

// Java code for forEach
```

```
// (Consumer action) in Java 8
import java.util.*;

class GFG {

   // Driver code
   public static void main(String[] args)
   {

      // Creating a list of Integers
      List<Integer> list = Arrays.asList(2, 4, 6, 8, 10);

      // Using forEach(Consumer action) to
      // print the elements of stream
      // in reverse order
      list.stream().sorted(Comparator.reverseOrder()).forEach(System.out::println);
   }
}

// Java code for forEach
// (Consumer action) in Java 8
import java.util.*;

class GFG {

   // Driver code
   public static void main(String[] args)
   {
```

```
        // Creating a list of Strings
        List<String> list = Arrays.asList("GFG", "Geeks",
                                  "for", "GeeksforGeeks");

        // Using forEach(Consumer action) to
        // print the elements of stream
        list.stream().forEach(System.out::println);
    }
}

// Java code for forEach
// (Consumer action) in Java 8
import java.util.*;
import java.util.stream.Stream;

class GFG {

    // Driver code
    public static void main(String[] args)
    {

        // Creating a Stream of Strings
        Stream<String> stream = Stream.of("GFG", "Geeks",
                                  "for", "GeeksforGeeks");

        // Using forEach(Consumer action) to print
        // Character at index 1 in reverse order
```

```
        stream.sorted(Comparator.reverseOrder())
            .flatMap(str -> Stream.of(str.charAt(1)))
            .forEach(System.out::println);
    }
}

// Java code for forEachOrdered
// (Consumer action) in Java 8
import java.util.*;

class GFG {

    // Driver code
    public static void main(String[] args) {

    // Creating a list of Integers
    List<Integer> list = Arrays.asList(10, 19, 20, 1, 2);

    // Using forEachOrdered(Consumer action) to
    // print the elements of stream in encounter order
    list.stream().forEachOrdered(System.out::println);



}
}

// Java code for forEachOrdered
// (Consumer action) in Java 8
```

```
import java.util.*;

class GFG {

    // Driver code
    public static void main(String[] args) {

    // Creating a list of Strings
    List<String> list = Arrays.asList("GFG", "Geeks",
                                      "for", "GeeksforGeeks");

    // Using forEachOrdered(Consumer action) to
    // print the elements of stream in encounter order
    list.stream().forEachOrdered(System.out::println);



}
}

// Java code for forEachOrdered
// (Consumer action) in Java 8
import java.util.*;
import java.util.stream.Stream;



class GFG {

    // Driver code
```

```
    public static void main(String[] args) {

    // Creating a Stream of Strings
    Stream<String> stream = Stream.of("GFG", "Geeks",
                          "for", "GeeksforGeeks");

    // Using forEachOrdered(Consumer action)
    stream.flatMap(str-> Stream.of(str.charAt(2)))
        .forEachOrdered(System.out::println);



}
}

// Java code for Stream findAny()
// which returns an Optional describing
// some element of the stream, or an
// empty Optional if the stream is empty.
import java.util.*;

class GFG {
    // Driver code
    public static void main(String[] args)
    {
        // Creating a List of Integers
        List<Integer> list = Arrays.asList(2, 4, 6, 8, 10);

        // Using Stream findAny() to return
```

```
        // an Optional describing some element
        // of the stream
        Optional<Integer> answer = list.stream().findAny();

        // if the stream is empty, an empty
        // Optional is returned.
        if (answer.isPresent()) {
            System.out.println(answer.get());
        }
        else {
            System.out.println("no value");
        }
    }
}

// Java code for Stream findAny()
// which returns an Optional describing
// some element of the stream, or an
// empty Optional if the stream is empty.
import java.util.*;

class GFG {
    // Driver code
    public static void main(String[] args)
    {

        // Creating a List of Strings
        List<String> list = Arrays.asList("Geeks", "for",
```

```
                                "GeeksQuiz", "GFG");

        // Using Stream findAny() to return
        // an Optional describing some element
        // of the stream
        Optional<String> answer = list.stream().findAny();

        // if the stream is empty, an empty
        // Optional is returned.
        if (answer.isPresent()) {
            System.out.println(answer.get());
        }
        else {
            System.out.println("no value");
        }
    }
}



// Java code for Stream findAny()
// which returns an Optional describing
// some element of the stream, or an
// empty Optional if the stream is empty.
import java.util.OptionalInt;
import java.util.stream.IntStream;

class GFG {
    // Driver code
```

```
    public static void main(String[] args)
    {

        // Creating an IntStream
        IntStream stream
            = IntStream.of(4, 5, 8, 10, 12, 16).parallel();

        // Using Stream findAny().
        // Executing the source code multiple times
        // may not return the same result.
        // Every time you may get a different
        // Integer which is divisible by 4.
        stream = stream.filter(i -> i % 4 == 0);

        OptionalInt answer = stream.findAny();
        if (answer.isPresent()) {
            System.out.println(answer.getAsInt());
        }
    }
}

// Java code for Stream anyMatch
// (Predicate predicate) to check whether
// any element of this stream match
// the provided predicate.
import java.util.*;

class GFG {
```

```
    // Driver code
    public static void main(String[] args) {

    // Creating a list of Integers
    List<Integer> list = Arrays.asList(3, 4, 6, 12, 20);

    // Stream anyMatch(Predicate predicate)
    boolean answer = list.stream().anyMatch(n
                -> (n * (n + 1)) / 4 == 5);

    // Displaying the result
    System.out.println(answer);
}
}

// Java code for  Stream anyMatch
// (Predicate predicate) to check whether
// any element of this stream match
// the provided predicate.
import java.util.stream.Stream;

class GFG {

    // Driver code
    public static void main(String[] args)
    {
```

```
        // Creating a Stream of Strings
        Stream<String> stream = Stream.of("Geeks", "fOr",
                                  "GEEKSQUIZ", "GeeksforGeeks");

        // Check if Character at 1st index is
        // UpperCase in any string or not using
        // Stream anyMatch(Predicate predicate)
        boolean answer = stream.anyMatch(str -> Character.isUpperCase(str.charAt(1)));

        // Displaying the result
        System.out.println(answer);
    }
}

// Java code for Stream allMatch
// (Predicate predicate) to check whether
// all elements of this stream match
// the provided predicate.
import java.util.*;

class GFG {

    // Driver code
    public static void main(String[] args) {

    // Creating a list of Integers
    List<Integer> list = Arrays.asList(3, 4, 6, 12, 20);
```

```
    // Check if all elements of stream
    // are divisible by 3 or not using
    // Stream allMatch(Predicate predicate)
    boolean answer = list.stream().allMatch(n-> n % 3 ==0);

    // Displaying the result
    System.out.println(answer);
}
}

// Java code for Stream allMatch
// (Predicate predicate) to check whether
// all elements of this stream match
// the provided predicate.
import java.util.stream.Stream;

class GFG {

    // Driver code
    public static void main(String[] args) {

    // Creating a Stream of Strings
    Stream<String> stream = Stream.of("Geeks", "for",
                    "GeeksQuiz", "GeeksforGeeks");

    // Check if all elements of stream
    // have length greater than 2 using
    // Stream allMatch(Predicate predicate)
```

```
    boolean answer = stream.allMatch(str -> str.length() > 2);

    // Displaying the result
    System.out.println(answer);
}
}

// Java code for Stream allMatch
// (Predicate predicate) to check whether
// all elements of this stream match
// the provided predicate.
import java.util.stream.Stream;

class GFG {

    // Driver code
    public static void main(String[] args) {

    // Creating a Stream of Strings
    Stream<String> stream = Stream.of("Geeks", "for",
                    "GeeksQuiz", "GeeksforGeeks");

    // Check if Character at 1st index is
    // UpperCase in all strings or not using
    // Stream allMatch(Predicate predicate)
    boolean answer = stream.allMatch(str-> Character
                    .isUpperCase(str.charAt(1)));
```

```
    // Displaying the result
    System.out.println(answer);
}
}

// In case we want multiple function to be done.

import java.util.stream.IntStream;

public class MultipleStreamFunction {

    public static void main(String[] args) {

        final String sample = "Om Sarve Bhavantu Sukhinah";

        // converting to Ascii
        IntStream intstreams = sample.chars();

        // All match to check if all Ascii value greater than 100
        boolean answer = intstreams.allMatch(c -> c > 100);
        System.out.println(answer);

        // Need to initialize the stream again
        // to avoid runtime exception
        intstreams = sample.chars();
        // All match to check if all Ascii value greater than 31

        answer = intstreams.allMatch(c -> c > 31);
```

```
        System.out.println(answer);

    }
}

// Java Program to get a Stream Consisting of the Elements
// of Stream that Matches Given Predicate for Stream filter
// (Predicate predicate)

// Importing required classes
import java.util.*;

// Class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        // Creating a list of Integers
        List<Integer> list = Arrays.asList(3, 4, 6, 12, 20);

        // Getting a stream consisting of the
        // elements that are divisible by 5
        // Using Stream filter(Predicate predicate)
        list.stream()
            .filter(num -> num % 5 == 0)
            .forEach(System.out::println);
```

```
    }
}

// Java Program to Get Stream Consisting of Elements
// of Stream that Matches Given Predicate
// for Stream Filter (Predicate predicate)

// Importing required classes
import java.util.stream.Stream;

// Class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {
        // Creating a stream of strings
        Stream<String> stream = Stream.of(
            "Geeks", "fOr", "GEEKSQUIZ", "GeeksforGeeks");

        // Getting a stream consisting of the
        // elements having UpperCase Character
        // at custom index say be it '1'
        // using Stream filter(Predicate predicate)
        stream
            .filter(
                str -> Character.isUpperCase(str.charAt(1)))
            .forEach(System.out::println);
```

```
    }
}
// Java Program to Get a Stream Consisting ofElements
//  of Stream that Matches Given predicate
// for Stream filter (Predicate predicate)

// Importing required classes
import java.util.stream.Stream;

// Class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        // Creating a stream of strings
        Stream<String> stream = Stream.of(
            "Geeks", "foR", "GeEksQuiz", "GeeksforGeeks");

        // Getting a stream consisting of the
        // elements ending with 's'
        // using Stream filter(Predicate predicate)
        stream.filter(str -> str.endsWith("s"))
            .forEach(System.out::println);
    }
}
```